DEPARTMENT OF COMPUTER SCIENCE AND IT

MCA PROGRAMME

ACADEMIC YEAR: 2026-2027

ACTIVITY – 2

Data Storytelling Through Visualization

"When does Montesinho burn — and what can a forest agency do about it?"

PREDICTIVE ANALYTICS AND DATA VISUALIZATION

25MCAAI302

SEM: III SPECIALIZATION: MCA-AIML

FACULTY NAME:

Prof. Rahul Pawar

Assistant Prof., Dept. of CS & IT.

STUDENT NAME: Anoon M R

USN: 25MCAR0179

Certificate

This is to certify that **Anoon M R (25MCAR0179)** satisfactorily completed the Activity-2 prescribed by the Jain (Deemed-to-be University) for the semester III M.C.A course in the year 2026-2027.

Date: /11/2026

Signature of Student

Signature of Faculty Incharge

Table of Contents

1. Problem Statement Explanation 4
1.1 Background 4
1.2 Problem statement 4
1.3 Objectives (from the course handbook, CA3) 4
1.4 The questions the story answers 4
2. Dataset Explanation 5
2.1 Source and collection 5
2.2 Attributes 5
2.3 Key characteristics 5
2.4 Why this dataset fits the problem 5
3. Solution Explanation 6
3.1 Solution overview 6
3.2 Data preparation 6
3.3 Act 1 — WHEN: the calendar of fire 7
3.4 Act 2 — WHY: associations among the weather variables 9
3.5 Act 3 — WHERE: hotspots on the park map 11
3.6 Interactive dashboard (Python — Plotly Dash) 12
3.7 Predictive model — early-warning risk tiers 12
3.8 Actionable insights 14
4. Screenshots 15
4.1 Dashboard — overview (all data) 15
4.2 Dashboard — filtered view (Summer weekends) 17
4.3 Dashboard — early-warning risk check 17
4.4 Jupyter notebook — data-quality checks 18
4.5 Jupyter notebook — cleaning and feature engineering 19
4.6 Jupyter notebook — model evaluation 20
5. Conclusion 20
5.1 Summary of findings 20
5.2 Limitations 20
5.3 Future scope 21
6. References 21
Appendix — Submission files 21

1. Problem Statement Explanation

1.1 Background

Wildfires destroy forests, wildlife habitat and property, and Mediterranean parks such as Montesinho Natural Park in north-east Portugal burn almost every summer. The agency that manages the park has limited fire crews, vehicles and surveillance hours. Today these resources are largely deployed reactively — after a fire is reported. To prevent damage instead of only responding to it, the agency needs to understand the pattern behind past fires and present it in a form that decision-makers can read at a glance.

1.2 Problem statement

Using historical weather and fire records from Montesinho Natural Park, find out **when**, **why** and **where** forest fires occur and cause the most damage, communicate these findings through clear visualizations and an interactive dashboard, and turn them into **actionable recommendations** and an **early-warning risk indicator** for the forest agency.

The problem is a **data-storytelling** problem: the numbers already exist, but in raw form (513 rows × 13 columns) they do not tell an officer what to do. The challenge is to choose visual forms that reveal the pattern honestly — including what the data **cannot** tell us.

1.3 Objectives (from the course handbook, CA3)

- Create **at least three visualizations** (Scatter Plot, Treemap, Correlogram, Mosaic Plot, etc.) — this report uses ten.
- Design a **simple dashboard** using Tableau / Power BI / Python — this activity uses **Python (Plotly Dash)**.
- **Interpret** the visualizations and present **actionable insights**.
- Deliverables: **Dashboard, Report and Presentation**.

1.4 The questions the story answers

Act	Question	Visualizations used
1. When	When do fires happen, and when does the damage happen?	Proportion bar, concentration curve, treemap, mosaic plot, small multiples
2. Why	Which weather conditions travel together with fire?	Correlogram, scatter plot, PCA biplot (dimension reduction), parallel sets
3. Where	Are some parts of the park hotspots?	Spatial grid heatmaps
4. What next	Can we turn this into an early-warning tool?	Model comparison, risk tiers, confusion matrix, feature importance

2. Dataset Explanation

2.1 Source and collection

The data is the **Forest Fires dataset** from the UCI Machine Learning Repository, published by Cortez & Morais (2007). It was collected in **Montesinho Natural Park** (Trás-os-Montes, north-east Portugal) between January 2000 and December 2003. Each row is one **day-and-location observation**: the park map is divided into a 9 × 9 grid, and for each recorded event the grid cell, date, weather readings, Fire Weather Index (FWI) codes and the burned area were stored. The raw file has **517 rows and 13 columns**. It is an open dataset in the climate / environment domain, as required by the activity.

2.2 Attributes

Attribute	Meaning	Type	Range in data
X, Y	Grid coordinates inside the park map (9 × 9 grid)	Spatial (integer)	X 1–9, Y 2–9
month, day	Month of the year and day of the week	Categorical	jan–dec, mon–sun
FFMC	Fine Fuel Moisture Code (FWI system) — dryness of surface litter	Numeric index	18.7 – 96.2
DMC	Duff Moisture Code — dryness of loosely compacted organic layers	Numeric index	1.1 – 291.3
DC	Drought Code — long-term dryness of deep organic layers	Numeric index	7.9 – 860.6
ISI	Initial Spread Index — expected rate of fire spread	Numeric index	0 – 56.1
temp	Temperature (°C)	Numeric	2.2 – 33.3
RH	Relative humidity (%)	Numeric	15 – 100
wind	Wind speed (km/h)	Numeric	0.4 – 9.4
rain	Outside rain (mm/m²)	Numeric	0 – 6.4
area	Burned area of the forest (hectares) — the outcome	Numeric (target)	0 – 1090.8

2.3 Key characteristics

- **Outcome:** 269 of 513 clean observations (52.4%) had a fire (area > 0); the rest burned nothing.
- **Extremely skewed area:** median fire 6.36 ha but maximum 1,090.8 ha — a few fires dominate the total of 6,613 ha.
- **Uneven sampling over the year:** August and September alone hold 69% of the rows, while 3 months have fewer than 5 observations.
- **Correlated inputs:** the four FWI codes are computed from the same weather history, so they move together (see the correlogram in Section 3.4).
- **Clean structure:** no missing values; only 4 exact duplicate rows.

2.4 Why this dataset fits the problem

It contains exactly the three ingredients the story needs — **time** (month, day), **conditions** (weather and FWI codes) and **place** (grid cell) — together with a measurable outcome (burned area). It mixes categorical, numeric and spatial variables, which allows every visualization family in Module 5 of the syllabus: proportions, nested proportions, associations and dimension reduction. The same dataset was used in Activity 1, which allowed Activity 2 to go deeper into storytelling rather than starting from scratch (reuse was approved by the faculty).

3. Solution Explanation

3.1 Solution overview

The solution is a five-step pipeline implemented in Python. Every number in this report is produced by the notebook and saved to outputs/metrics.json, so the report, dashboard and presentation always agree.

Step	What was done	Tools	Output
1. Prepare	Quality checks, duplicate removal, feature engineering	pandas, NumPy	forestfires_clean.csv
2. Visualize	Ten charts in three acts (When / Why / Where)	Matplotlib, Plotly	figures/*.png
3. Dashboard	Interactive dashboard with filters, KPIs, six charts and a risk check	Plotly Dash	dashboard/app.py, Wildfire_Dashboard.html
4. Model	Compare four classifiers with cross-validation; build risk tiers	scikit-learn	Random Forest risk tiers
5. Act	Translate findings into recommendations	—	Section 3.8

A single visual identity is used throughout: a grey-to-dark-orange ordered ramp for fire size, blue vs orange for two-group comparisons, and a blue–grey–red diverging scale for correlations. The palette was checked with a colour-blindness validator so that every chart remains readable for colour-vision-deficient readers.

3.2 Data preparation

3.2.1 Data-quality checks

Check	Result	Decision
Missing values	0 in every column	No imputation needed
Exact duplicate rows	4 duplicates	Removed → 513 rows remain
Domain/range checks (RH 0–100, FFMC 0–101, area ≥ 0, valid grid, valid month/day labels)	All PASS	No corrections needed
Outliers (IQR rule)	In every numeric column — most in area (63), FFMC (53), ISI (23)	Kept — they are real events (rainy days, windy days, catastrophic fires), not recording errors
Skewness of area	Median fire 6.36 ha, maximum 1,090.8 ha	Added log(1 + area) for plotting and correlation

3.2.2 Feature engineering

- **season** — Winter (Dec–Feb), Spring (Mar–May), Summer (Jun–Aug), Autumn (Sep–Nov).
- **month_num, day_num** — calendar-ordered numeric codes (not alphabetical).
- **fire** — 1 if any area burned, else 0 (the model target).
- **size_class** — No fire, Small (< 1 ha), Medium (1–10 ha), Large (> 10 ha).
- **ffmc_band** — Dry fuel (FFMC ≥ 90) vs Moist fuel; **temp_band** — Cool (< 15 °C), Mild (15–22 °C), Hot (> 22 °C).
- **period** — Peak (Aug–Sep) vs Rest of year; **day_type** — Weekday vs Weekend; **log_area** = log(1 + area).

After cleaning: **513 observations**, of which **269 (52.4%) had a fire**, burning **6,613 hectares** in total.

3.3 Act 1 — WHEN: the calendar of fire

3.3.1 Proportions as parts of the total

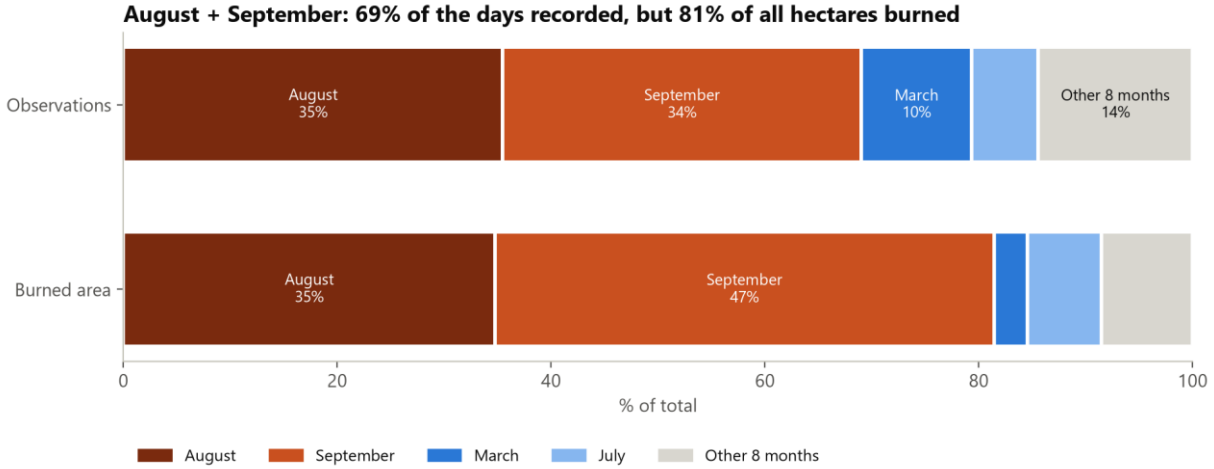


Figure 1: Share of observations vs share of burned area by month (100% stacked bars)

Interpretation: August and September together account for **69% of all observations** but **81% of all burned hectares**. September alone holds 34% of the observations yet 47% of the damage — fires in September are larger than their share of days would suggest. March has 10% of observations but only 3% of the burned area.

3.3.2 Concentration of damage

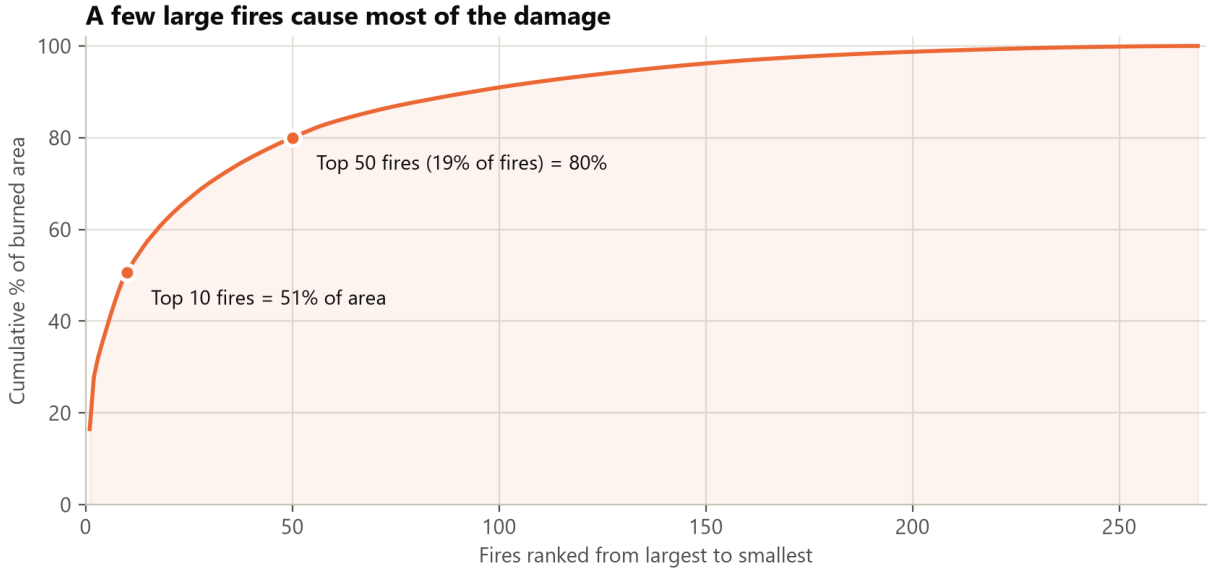


Figure 2: Cumulative share of burned area, fires ranked from largest to smallest

Interpretation: the damage is extremely concentrated. The **10 largest fires cause 51%** of all burned area, and the top 50 fires (only 19% of all fires) cause 80%. The typical (median) fire burns just 6.36 ha. The practical goal is therefore to stop the few fires that grow large.

3.3.3 Treemap — nested proportions (season → month)

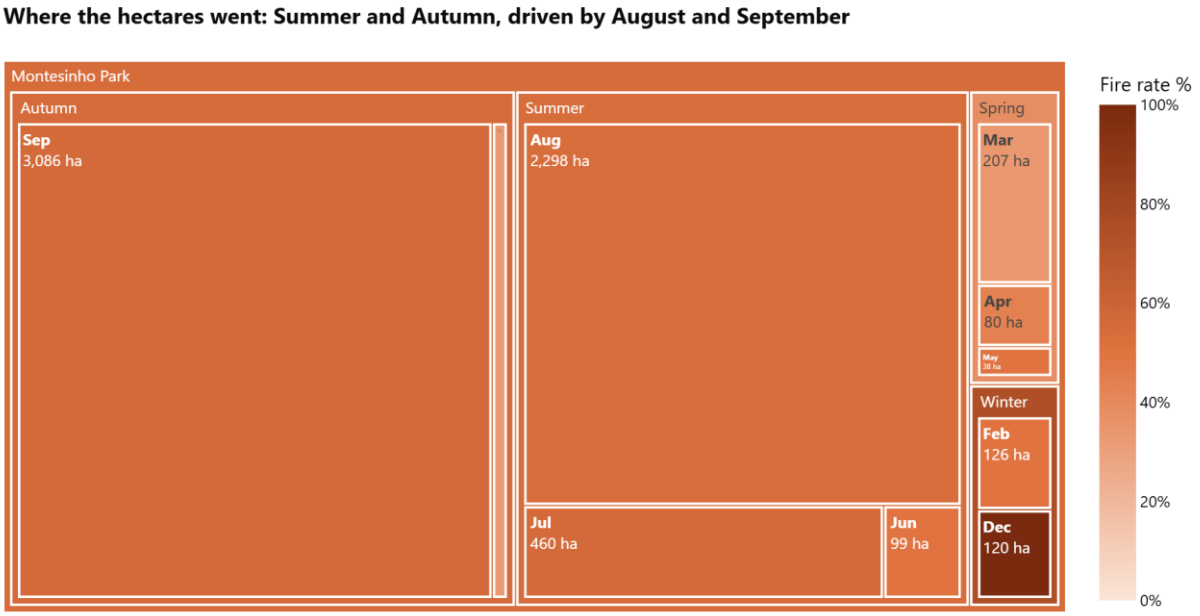


Figure 3: Treemap: rectangle size = hectares burned, colour = fire rate (share of observations with a fire)

Interpretation: the nested view shows that Autumn and Summer hold almost all of the burned area, and inside them a single month dominates each season — September (3,086 ha) and August (2,298 ha). Spring and Winter are small rectangles. The colour shows fire rates between about 33% and 56% in most months; the darkest cell (December) is based on only 9 observations, so it should not be over-interpreted.

3.3.4 Mosaic plot — season × fire-size class

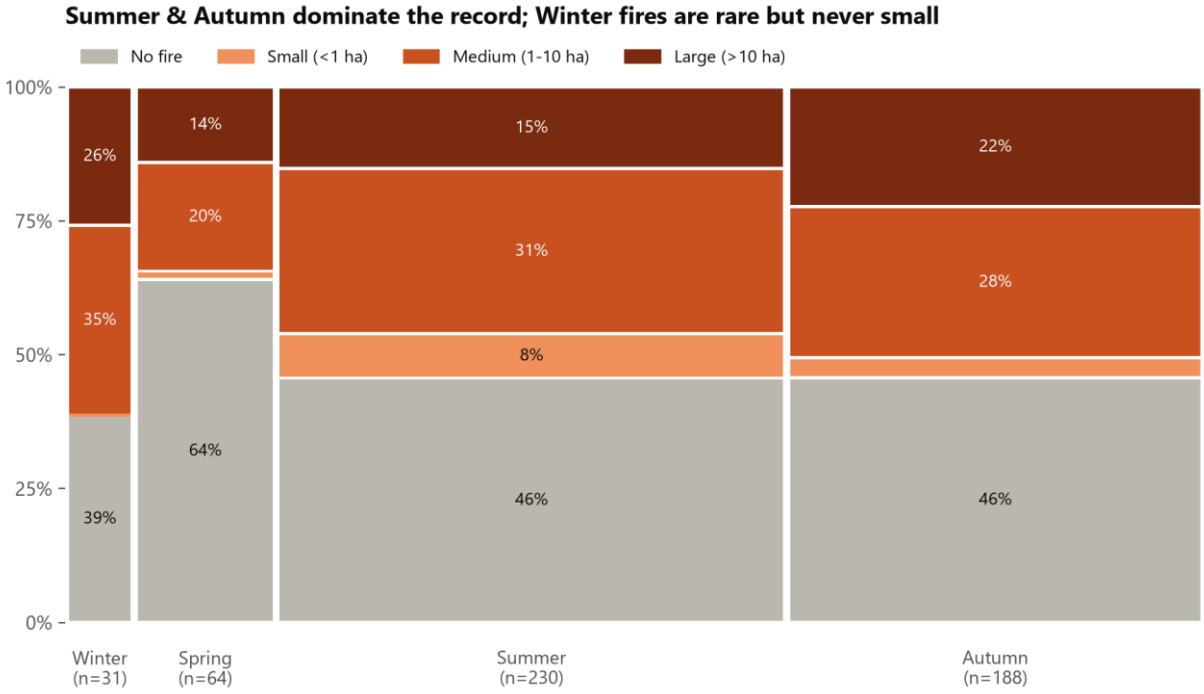


Figure 4: Mosaic plot: column width = share of observations per season; segment height = share of each fire-size class

Interpretation: the wide Summer and Autumn columns show that most of the record comes from those seasons. Within each season, Autumn has the highest share of large fires among the well-sampled seasons (22% vs 15% in Summer), and Spring has the most no-fire observations (64%). Winter is a thin column (only 31 observations), but none of its fires were small — when they happen they reach medium or large size.

3.3.5 Small multiples — the dry-season build-up

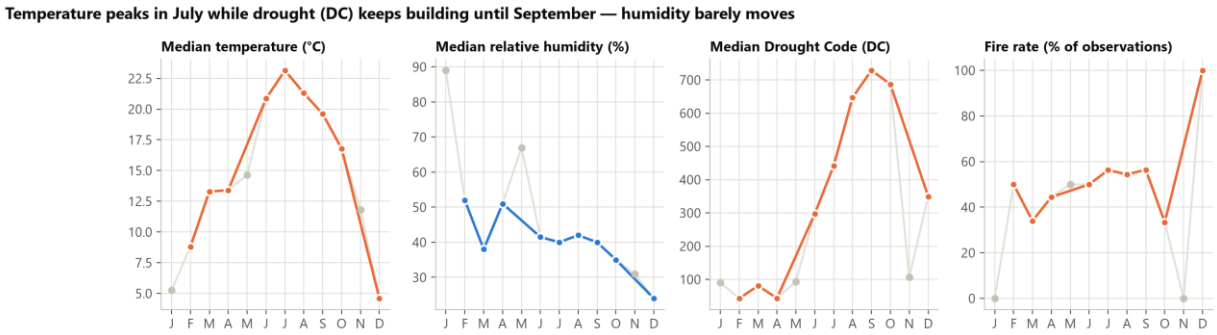


Figure 5: Monthly medians of temperature, humidity, Drought Code and the fire rate (faded points: < 5 observations)

Interpretation: each variable gets its own panel and its own scale (no misleading dual axis). Temperature peaks in July (≈ 23 °C) while the Drought Code keeps rising until September (median ≈ 730), because drought accumulates over weeks. Humidity changes much less (around 40% through the summer). Among the well-sampled months, the fire rate is highest from July to September (about 55%).

3.4 Act 2 — WHY: associations among the weather variables

3.4.1 Correlogram

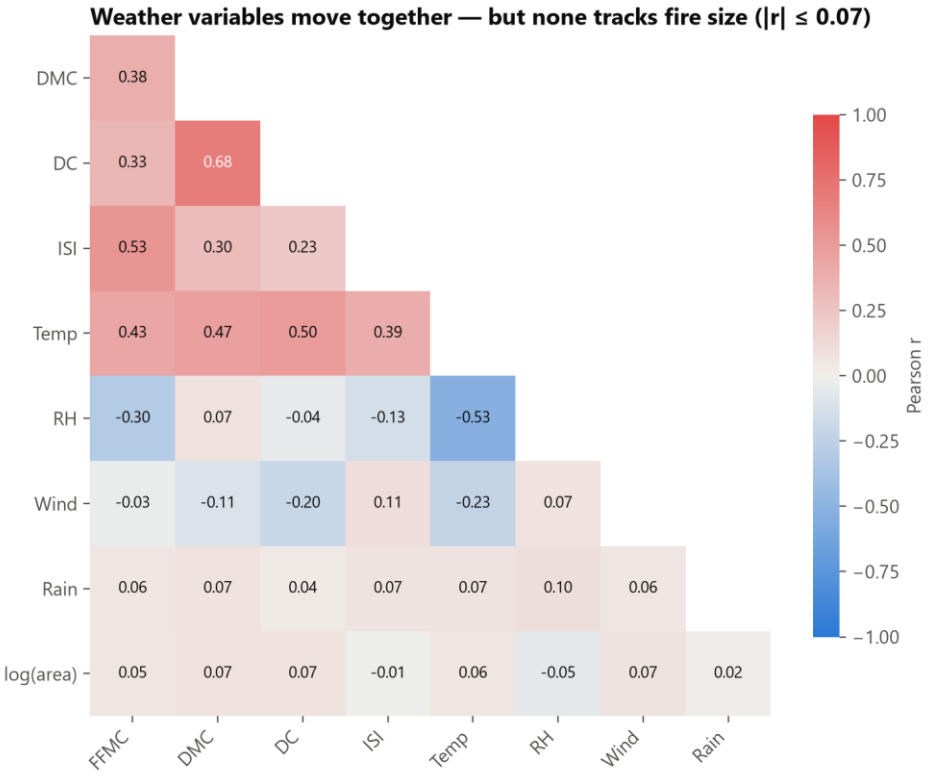


Figure 6: Correlogram (Pearson r) of the weather variables and log(1 + area)

Interpretation: the weather variables are strongly related to each other — DMC and DC (r = 0.68), temperature and humidity (r = −0.53), FFMC and ISI (r = 0.53). However, **no variable is meaningfully correlated with fire size — the strongest is only |r| = 0.07**. Weather sets the stage, but on its own it does not decide how big a fire becomes.

3.4.2 Scatter plot — temperature vs humidity

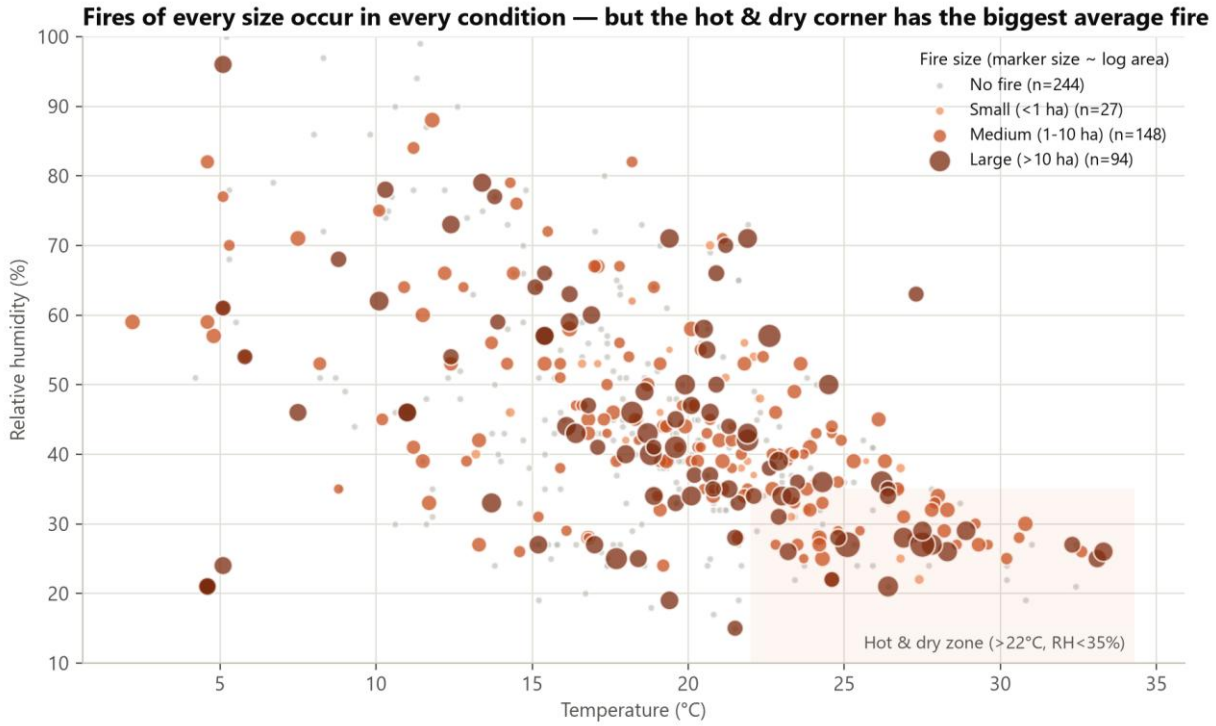


Figure 7: Scatter plot of temperature vs relative humidity; colour = fire-size class, marker size ∝ log(burned area)

Interpretation: fires of every size appear across the whole range of conditions, so there is no clean boundary. But the shaded **hot & dry corner (> 22 °C and RH < 35%)** stands out: its 87 observations have a fire rate of 59% and an **average fire of 30 ha**, against 51% and 9 ha on all other days.

3.4.3 Dimension reduction — PCA

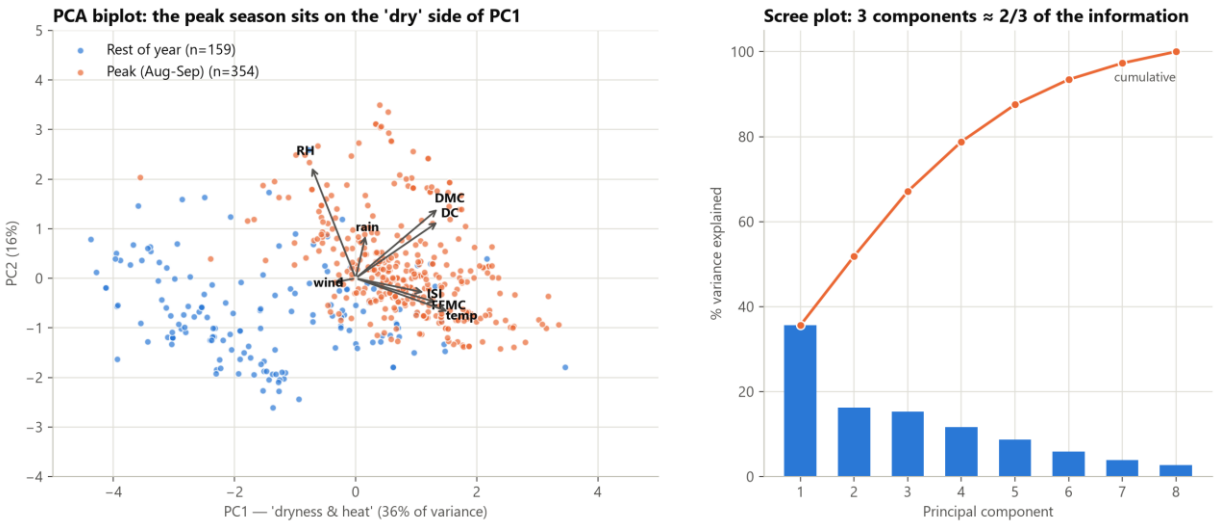


Figure 8: PCA biplot of the 8 standardized weather variables (left) and scree plot (right)

Interpretation: the first principal component explains 36% of the variance and loads on temperature, FFMC, ISI, DMC and DC — a 'dryness & heat' axis — while humidity points the opposite way. Peak-season observations (orange) sit on the dry/hot side, and the rest of the year (blue) on the cool/humid side. Three components keep about 67% of the information, so eight variables can be summarised by three.

3.4.4 Parallel sets — conditions flowing into outcomes

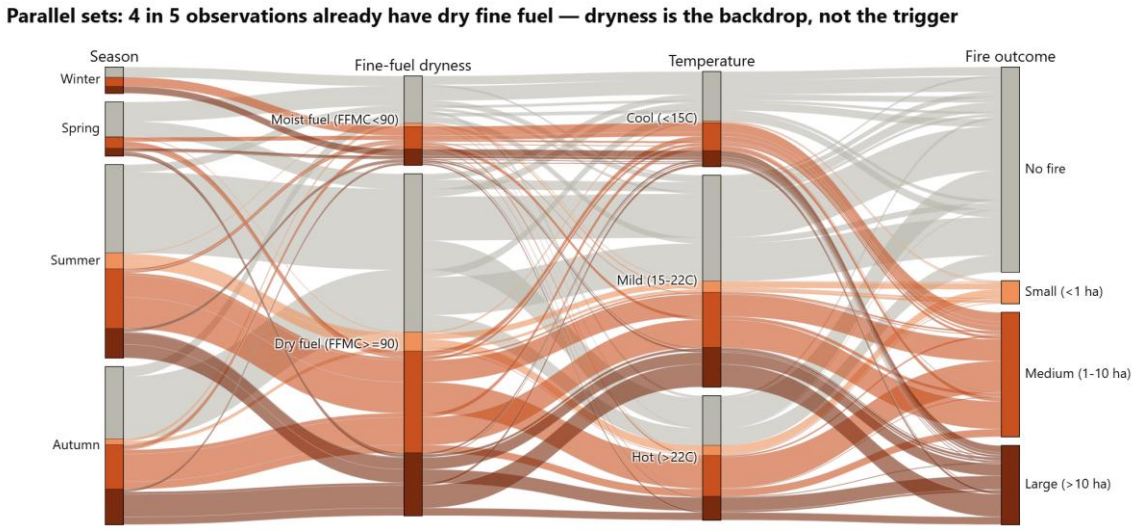


Figure 9: Parallel sets: Season → fine-fuel dryness → temperature band → fire outcome

Interpretation: 79% of observations already have dry fine fuel (FFMC ≥ 90), and ribbons from the dry-fuel bar flow into every outcome, including 'No fire'. Dry fuel is the **backdrop** of the fire season rather than the trigger: an ignition (often human) is still needed.

3.5 Act 3 — WHERE: hotspots on the park map

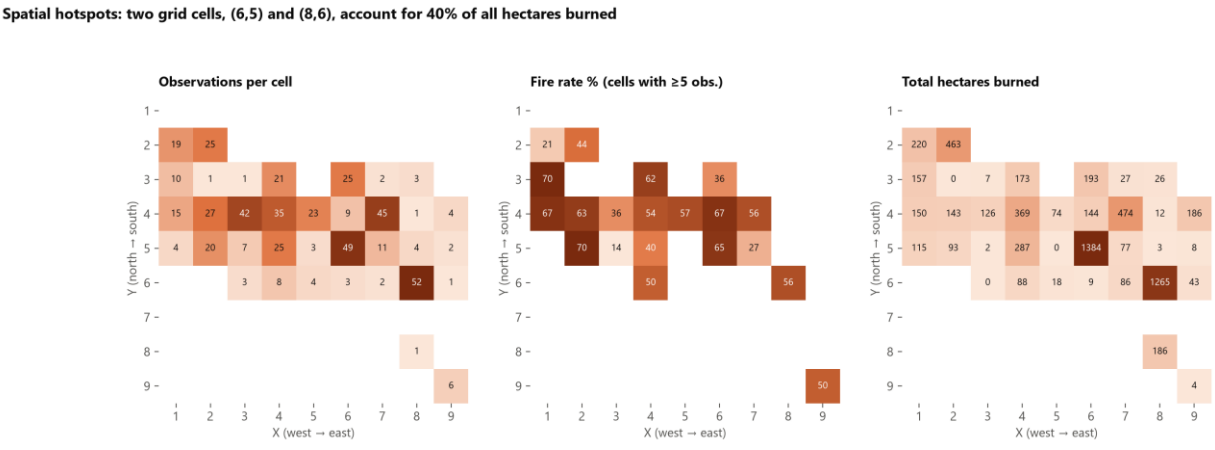


Figure 10: Park grid (X west→east, Y north→south): observations, fire rate and total hectares burned per cell

Interpretation: two cells — **(X=6, Y=5)** with 1,384 ha and **(X=8, Y=6)** with 1,265 ha — account for **40% of all hectares burned**. They are also two of the most frequently observed cells, so they are persistent problem areas.

3.6 Interactive dashboard (Python — Plotly Dash)

The dashboard brings the main charts together in one screen so that a forest officer can explore the data without writing code. Screenshots are given in Section 4.

- **One filter row** (season, month range, weekday/weekend) that updates every chart and KPI together.
- **KPI tiles:** observations, fire rate, hectares burned, largest fire, median temperature.
- **Six linked charts:** monthly stacked bars, treemap, scatter plot, park-grid heatmap, parallel sets and correlogram — all with hover tooltips.
- **Early-warning risk check:** choose month, grid cell and weather values; the Random Forest returns a Low / Moderate / High tier and the historical fire rate of that tier.
- **Shareable views:** filters can be set from the link, e.g. …/?season=Summer&day=Weekend.
- **Offline copy:** dashboard/Wildfire_Dashboard.html opens in any browser without Python.

How to run:

```
pip install dash plotly pandas scikit-learn
python dashboard/app.py
Open http://127.0.0.1:8050 in a browser
```

3.7 Predictive model — early-warning risk tiers

3.7.1 Setup

- **Target:** fire = 1 if any area burned (52% of observations — balanced classes).
- **Features (12):** X, Y, month, day, FFMC, DMC, DC, ISI, temperature, RH, wind, rain.
- **Models compared:** majority-class baseline, Logistic Regression, Decision Tree (depth 4), Random Forest (300 trees, min_samples_leaf = 5, balanced class weights).
- **Evaluation:** 5 × repeated stratified 5-fold cross-validation (25 fits per model) with ROC-AUC, so the result does not depend on one lucky train/test split; plus a 25% hold-out set for the confusion matrix.

3.7.2 Results

Model	CV ROC-AUC (mean ± sd)
Baseline (always majority)	0.500 ± 0.000
Logistic Regression	0.536 ± 0.047
Decision Tree (depth 4)	0.561 ± 0.045
Random Forest (300 trees)	0.602 ± 0.044

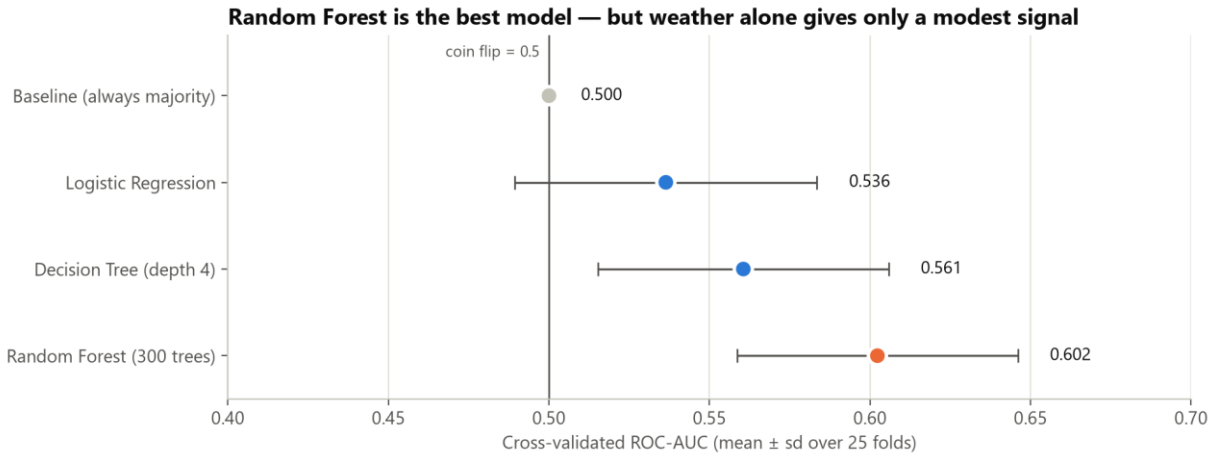


Figure 11: Cross-validated ROC-AUC of the four models (0.5 = coin flip)

The **Random Forest is the best model (ROC-AUC 0.60)**. On the single hold-out split it scores 0.68, which shows how much one split can flatter a model — the cross-validated figure is the honest one.

Hold-out metric	No fire	Fire
Precision	68%	67%
Recall	59%	75%
F1-score	0.63	0.71
Accuracy	67.4%	

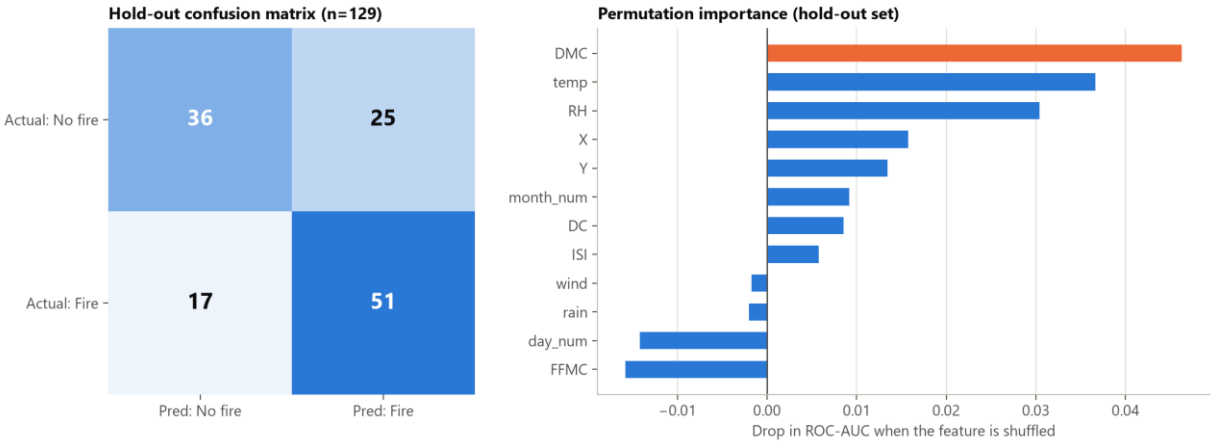


Figure 12: Hold-out confusion matrix (left) and permutation importance (right)

The confusion matrix shows 51 fires correctly flagged and 17 missed, with 25 false alarms. Permutation importance ranks **DMC, temp and RH** as the most useful inputs — the moisture/drought and heat variables, followed by location (X, Y).

3.7.3 From probabilities to risk tiers

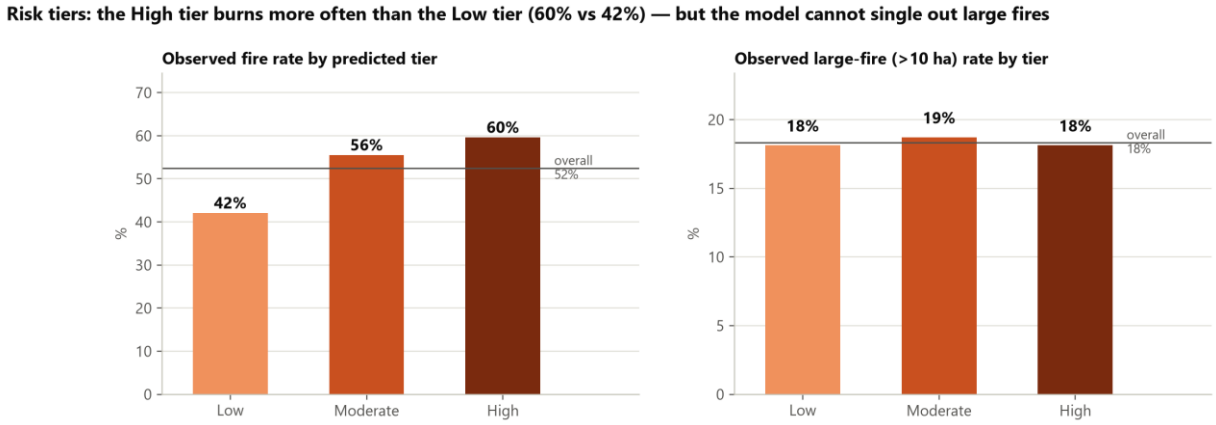


Figure 13: Observed fire rate and large-fire rate in each predicted risk tier (out-of-fold predictions)

Every observation was scored by a model that never saw it in training, then split into three equal tiers. The **High tier burned 60% of the time vs 42% in the Low tier** — a useful ranking for planning patrols. However, the share of large fires is about 18% in every tier: the model can rank how likely a fire is, but not how big it will grow.

3.8 Actionable insights

#	What the data shows	Recommended action
1	Aug–Sep = 69% of observations and 81% of burned hectares.	Concentrate crews, patrols and aerial surveillance in August–September; schedule leave and maintenance outside this window.
2	The 10 largest fires ≈ 51% of all burned area.	Invest in fast initial attack so that the few fires that would become large are stopped early.
3	Drought Code keeps building until September (median ≈ 650–730 in Aug–Sep vs < 100 in spring).	Use the Fire Weather Index codes (DC, DMC) as a seasonal readiness trigger, e.g. raise readiness when DC exceeds about 600.
4	Hot & dry days (> 22 °C, RH < 35%) average 30 ha per observation vs 9 ha otherwise.	On hot & dry forecast days pre-position water tenders and restrict campfires and public access.
5	Fine fuel is dry on 79% of days, yet many of those days have no fire.	Combine weather with ignition information (roads, tourism, agricultural burning) — dryness alone is not enough.
6	Cells (6,5) and (8,6) hold 40% of burned hectares.	Place watch-towers / cameras and fuel breaks in those cells first.
7	Random Forest risk tiers: High 60% vs Low 42% fire rate.	Use the dashboard's risk check to rank days for patrol priority — not as a yes/no alarm.

4. Screenshots

The screenshots below were captured from the running dashboard (Plotly Dash, http://127.0.0.1:8050) and from the executed Jupyter notebook (Activity2_Data_Storytelling.ipynb).

4.1 Dashboard — overview (all data)

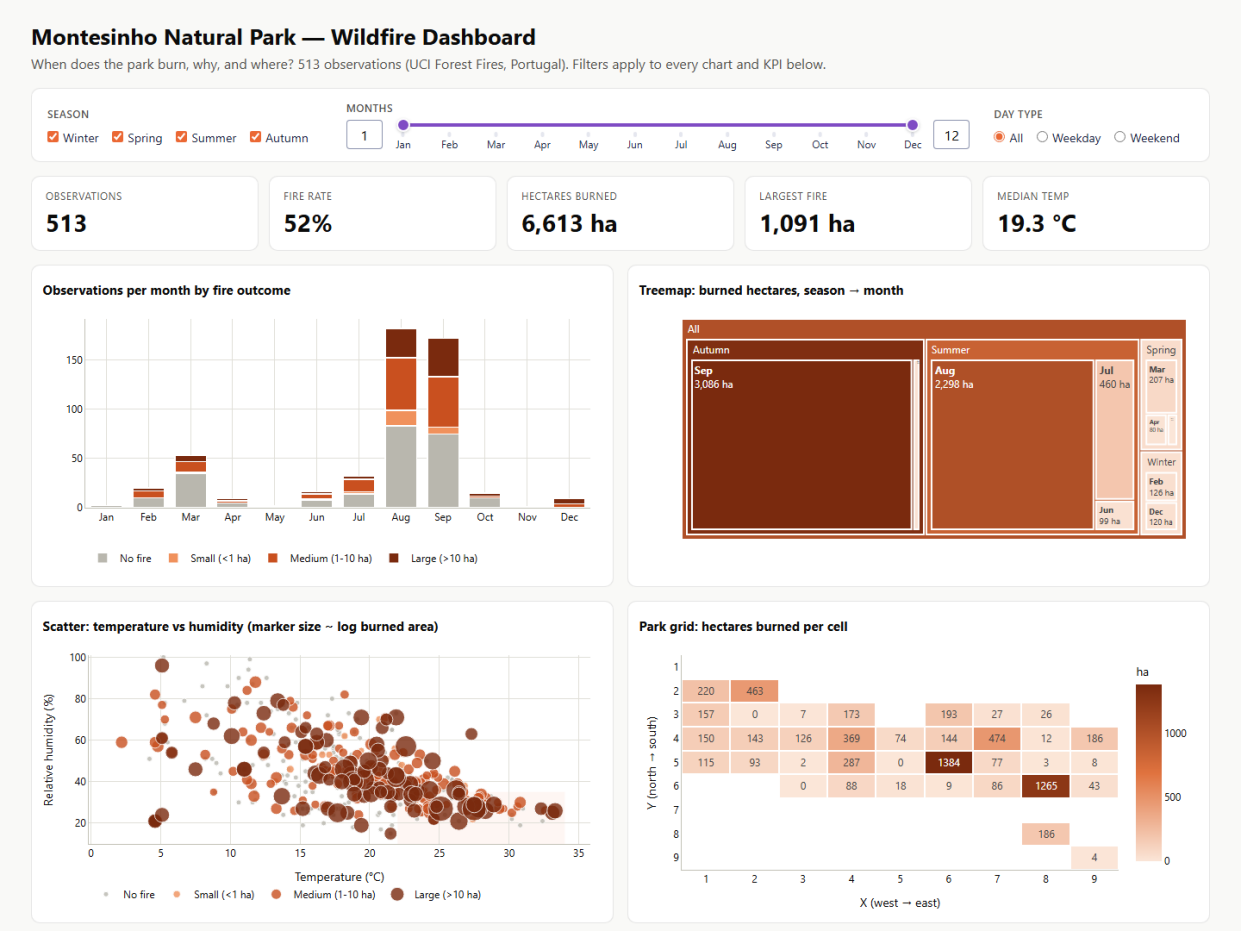


Figure 14: Dashboard overview — filter row, KPI tiles, monthly bars, treemap, scatter plot and park grid

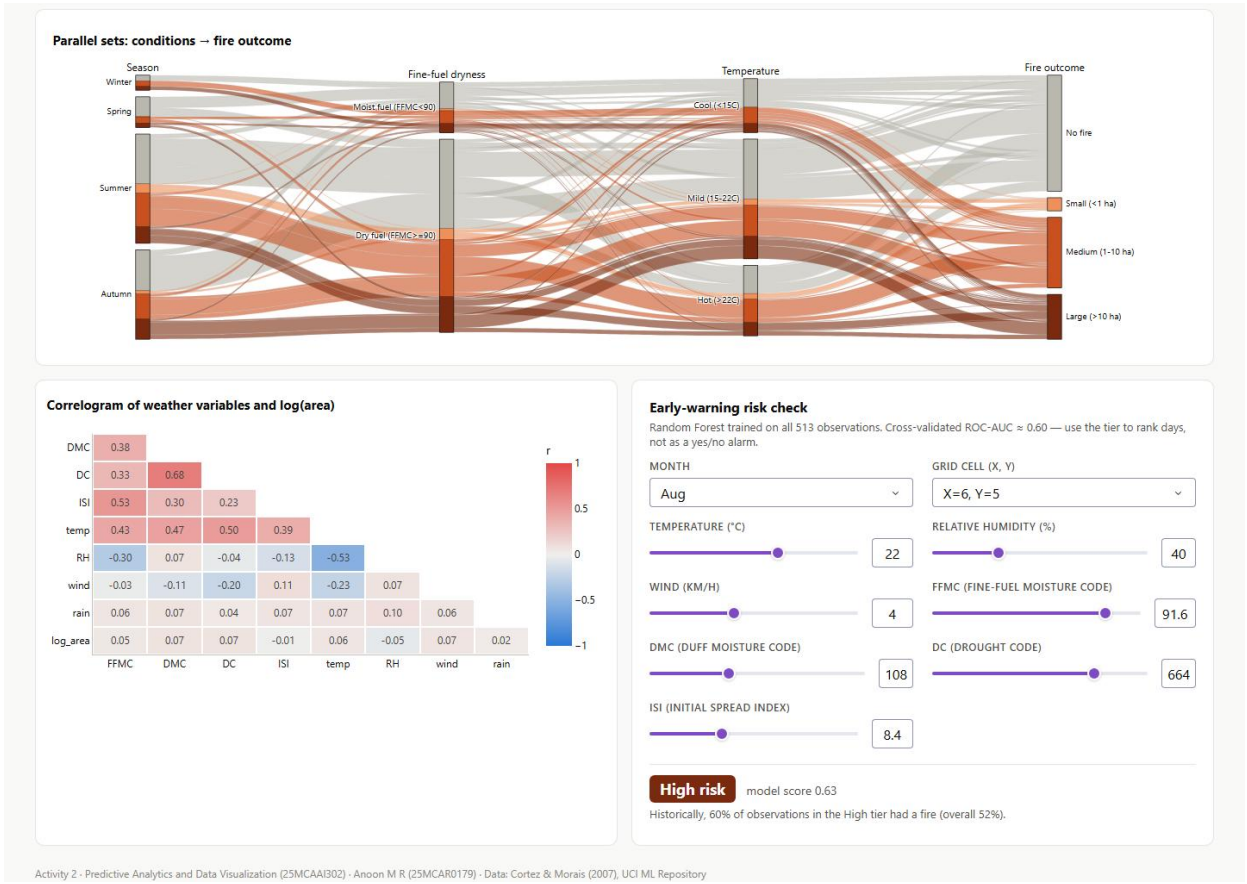


Figure 15: Dashboard overview (continued) — parallel sets, correlogram and the early-warning risk check

4.2 Dashboard — filtered view (Summer weekends)

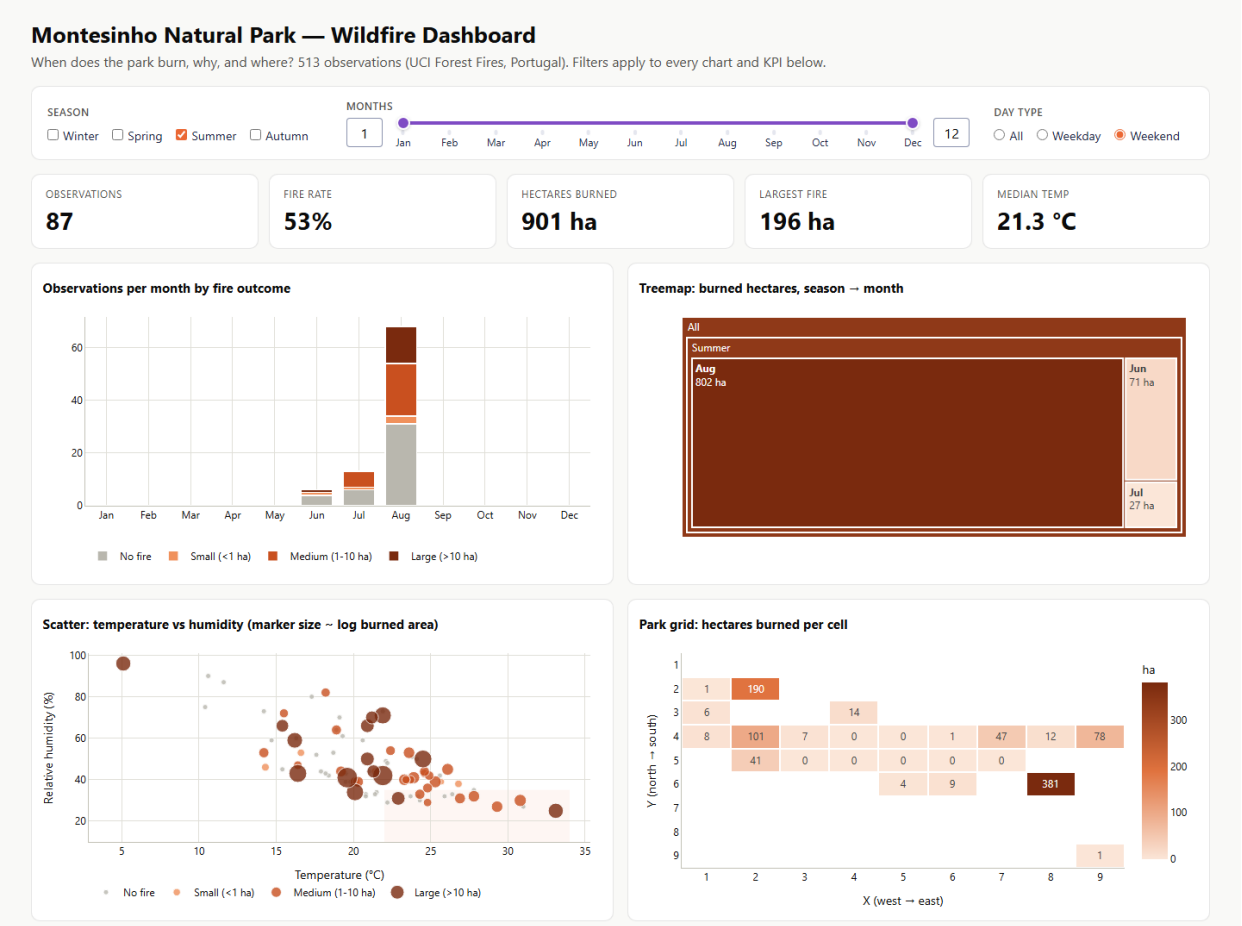


Figure 16: Season = Summer and Day type = Weekend: every KPI and chart updates to the selected slice

With the filter applied, the KPIs change to **87 observations**, a **53% fire rate** and **901 ha** burned, and the treemap shows that August dominates summer-weekend damage. This demonstrates that the filters drive every chart at once.

4.3 Dashboard — early-warning risk check

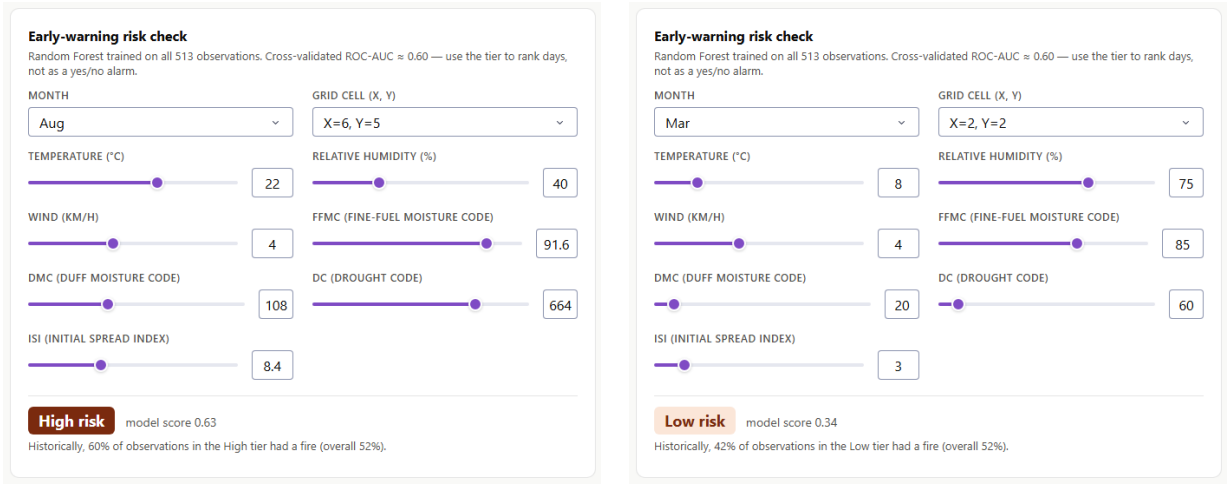


Figure 17: Risk check for a hot, dry August day in hotspot cell (6,5) → High risk (left), and a cool, humid March day in cell (2,2) → Low risk (right)

4.4 Jupyter notebook — data-quality checks

In [5]:
```
print("Missing values per column:\n", raw.isnull().sum().to_string(), "\n")
dups = raw[raw.duplicated(keep=False)]
print(f"Exact duplicate rows: {raw.duplicated().sum()} (shown below with their twins)")
dups
```

```
Missing values per column:
 X        0
Y        0
month    0
day      0
FFMC     0
DMC      0
DC       0
ISI      0
temp     0
RH       0
wind     0
rain     0
area     0

Exact duplicate rows: 4 (shown below with their twins)
```

Out[5]:

	X	Y	month	day	FFMC	DMC	DC	ISI	temp	RH	wind	rain	area
52	4	3	aug	wed	92.1	111.2	654.1	9.6	20.4	42	4.9	0.0	0.00
53	4	3	aug	wed	92.1	111.2	654.1	9.6	20.4	42	4.9	0.0	0.00
99	3	4	aug	sun	91.4	142.4	601.4	10.6	19.8	39	5.4	0.0	0.00
100	3	4	aug	sun	91.4	142.4	601.4	10.6	19.8	39	5.4	0.0	0.00
214	4	4	mar	sat	91.7	35.8	80.8	7.8	17.0	27	4.9	0.0	28.66
215	4	4	mar	sat	91.7	35.8	80.8	7.8	17.0	27	4.9	0.0	28.66
302	3	6	jun	fri	91.1	94.1	232.1	7.1	19.2	38	4.5	0.0	0.00
303	3	6	jun	fri	91.1	94.1	232.1	7.1	19.2	38	4.5	0.0	0.00

In [6]:
```
# Range checks against the documented domain of each variable (forestfires.names)
checks = {
    "RH within 0-100 %":           raw["RH"].between(0, 100).all(),
    "FFMC within 0-101":           raw["FFMC"].between(0, 101).all(),
    "Area non-negative":           (raw["area"] >= 0).all(),
    "X grid within 1-9":           raw["X"].between(1, 9).all(),
    "Y grid within 2-9":           raw["Y"].between(2, 9).all(),
    "Month labels valid":          raw["month"].isin(["jan","feb","mar","apr","may","jun","jul","aug","sep","oct","nov","dec"]).all(),
    "Day labels valid":            raw["day"].isin(["mon","tue","wed","thu","fri","sat","sun"]).all(),
}
for k, v in checks.items():
    print(f"  {'PASS' if v else 'FAIL'}  {k}")

# IQR outlier count per numeric column
num_cols = ["FFMC", "DMC", "DC", "ISI", "temp", "RH", "wind", "rain", "area"]
q1, q3 = raw[num_cols].quantile(0.25), raw[num_cols].quantile(0.75)
iqr = q3 - q1
outliers = ((raw[num_cols] < q1 - 1.5 * iqr) | (raw[num_cols] > q3 + 1.5 * iqr)).sum()
print("\nIQR outliers per column:\n", outliers.to_string())
```

```
  PASS  RH within 0-100 %
  PASS  FFMC within 0-101
  PASS  Area non-negative
  PASS  X grid within 1-9
  PASS  Y grid within 2-9
  PASS  Month labels valid
  PASS  Day labels valid

IQR outliers per column:
 FFMC    53
DMC     17
DC       8
ISI     23
temp     2
RH      12
wind    13
rain     8
area    63
```

Figure 18: Notebook: missing-value count, duplicate rows, range checks (all PASS) and IQR outlier counts

4.5 Jupyter notebook — cleaning and feature engineering

In [7]:
```
df = raw.drop_duplicates().reset_index(drop=True)

MONTHS = ["jan","feb","mar","apr","may","jun","jul","aug","sep","oct","nov","dec"]
DAYS = ["mon","tue","wed","thu","fri","sat","sun"]
SEASON = {"dec":"Winter","jan":"Winter","feb":"Winter","mar":"Spring","apr":"Spring","may":"Spring",
          "jun":"Summer","jul":"Summer","aug":"Summer","sep":"Autumn","oct":"Autumn","nov":"Autumn"}
SEASON_ORDER = ["Winter", "Spring", "Summer", "Autumn"]

df["month_num"]   = df["month"].map(lambda m: MONTHS.index(m) + 1)
df["month_lbl"]   = df["month"].str.capitalize()
df["day_num"]     = df["day"].map(lambda d: DAYS.index(d) + 1)
df["season"]      = pd.Categorical(df["month"].map(SEASON), SEASON_ORDER, ordered=True)
df["day_type"]    = np.where(df["day"].isin(["sat", "sun"]), "Weekend", "Weekday")
df["period"]      = np.where(df["month"].isin(["aug", "sep"]), "Peak (Aug-Sep)", "Rest of year")
df["fire"]        = (df["area"] > 0).astype(int)
df["size_class"] = pd.Categorical(pd.cut(df["area"], [-1, 0, 1, 10, np.inf], labels=SIZE_ORDER),
                                  SIZE_ORDER, ordered=True)
df["log_area"]    = np.log1p(df["area"])
# Danger bands from the Canadian Fire Weather Index system: FFMC >= 90 = fine fuels are very dry
df["ffmc_band"]   = np.where(df["FFMC"] >= 90, "Dry fuel (FFMC>=90)", "Moist fuel (FFMC<90)")
df["temp_band"]   = pd.cut(df["temp"], [-1, 15, 22, 40], labels=["Cool (<15C)", "Mild (15-22C)", "Hot (>22C)"])

print("Clean shape:", df.shape)
df.to_csv("data/forestfires_clean.csv", index=False)
df[["month", "season", "day_type", "area", "size_class", "fire", "ffmc_band", "temp_band"]].head(8)
```

```
Clean shape: (513, 24)
```

Out[7]:

	month	season	day_type	area	size_class	fire	ffmc_band	temp_band
0	mar	Spring	Weekday	0.0	No fire	0	Moist fuel (FFMC<90)	Cool (<15C)
1	oct	Autumn	Weekday	0.0	No fire	0	Dry fuel (FFMC>=90)	Mild (15-22C)
2	oct	Autumn	Weekend	0.0	No fire	0	Dry fuel (FFMC>=90)	Cool (<15C)
3	mar	Spring	Weekday	0.0	No fire	0	Dry fuel (FFMC>=90)	Cool (<15C)
4	mar	Spring	Weekend	0.0	No fire	0	Moist fuel (FFMC<90)	Cool (<15C)
5	aug	Summer	Weekend	0.0	No fire	0	Dry fuel (FFMC>=90)	Hot (>22C)
6	aug	Summer	Weekday	0.0	No fire	0	Dry fuel (FFMC>=90)	Hot (>22C)
7	aug	Summer	Weekday	0.0	No fire	0	Dry fuel (FFMC>=90)	Cool (<15C)

In [8]:
```
summary = {
    "rows_raw": int(len(raw)), "rows_clean": int(len(df)), "duplicates_removed": int(len(raw) - len(df)),
    "fires": int(df["fire"].sum()), "fire_rate": round(df["fire"].mean() * 100, 1),
    "total_area": round(df["area"].sum(), 1), "max_area": round(df["area"].max(), 1),
    "median_fire_area": round(df.loc[df.fire == 1, "area"].median(), 2),
}
print(json.dumps(summary, indent=2))
```

```
{
  "rows_raw": 517,
  "rows_clean": 513,
  "duplicates_removed": 4,
  "fires": 269,
  "fire_rate": 52.4,
  "total_area": 6613.4,
  "max_area": 1090.8,
  "median_fire_area": 6.36
}
```

Figure 19: Notebook: duplicate removal, engineered features and the cleaned-data summary

4.6 Jupyter notebook — model evaluation

In [19]:
```
FEATURES = ["X", "Y", "month_num", "day_num", "FFMC", "DMC", "DC", "ISI", "temp", "RH", "wind", "rain"]
Xf, yf = df[FEATURES], df["fire"]
models = {
    "Baseline (always majority)": DummyClassifier(strategy="most_frequent"),
    "Logistic Regression": make_pipeline(StandardScaler(), LogisticRegression(class_weight="balanced", max_iter=1000)),
    "Decision Tree (depth 4)": DecisionTreeClassifier(max_depth=4, min_samples_leaf=10, class_weight="balanced", random_state=42),
    "Random Forest (300 trees)": RandomForestClassifier(n_estimators=300, min_samples_leaf=5, class_weight="balanced", random_state=42, n_jobs=
}
rcv = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=42)
cv_res = {}
for name, m in models.items():
    s = cross_val_score(m, Xf, yf, cv=rcv, scoring="roc_auc")
    cv_res[name] = (s.mean(), s.std())
    print(f"{name:28s} ROC-AUC {s.mean():.3f} ± {s.std():.3f}")
```

```
Baseline (always majority)   ROC-AUC 0.500 ± 0.000
Logistic Regression          ROC-AUC 0.536 ± 0.047
Decision Tree (depth 4)      ROC-AUC 0.561 ± 0.045
Random Forest (300 trees)    ROC-AUC 0.602 ± 0.044
```

In [21]:
```
rf = models["Random Forest (300 trees)"]
oof = cross_val_predict(rf, Xf, yf, cv=StratifiedKFold(5, shuffle=True, random_state=42), method="predict_proba")[:, 1]
df["risk_prob"] = oof
df["risk_tier"] = pd.qcut(oof, 3, labels=["Low", "Moderate", "High"])
tier = df.groupby("risk_tier", observed=True).agg(n=("fire", "size"), fire_rate=("fire", "mean"),
                                                  large_rate=("area", lambda a: (a > 10).mean()),
                                                  burned_ha=("area", "sum"))
tier["fire_rate"] *= 100; tier["large_rate"] *= 100
print("Out-of-fold ROC-AUC:", round(roc_auc_score(yf, oof), 3))
print(tier.round(1))

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
tier_cols = ["#f0915c", "#c9501f", "#7a2a0e"]
for ax, col, title, fmt in [(axes[0], "fire_rate", "Observed fire rate by predicted tier", "{:.0f}%"),
                            (axes[1], "large_rate", "Observed large-fire (>10 ha) rate by tier", "{:.0f}%")]:
    ax.bar(tier.index.astype(str), tier[col], color=tier_cols, width=0.55)
    for i, v in enumerate(tier[col]):
        ax.text(i, v + 1, fmt.format(v), ha="center", fontsize=11, fontweight="bold", color=INK)
    ax.axhline(df["fire"].mean() * 100 if col == "fire_rate" else (df.area > 10).mean() * 100, color=INK2, lw=1)
    ax.set_title(title, fontsize=11.5); ax.set_ylabel("%"); ax.grid(axis="x", visible=False)
    ax.set_ylim(0, max(tier[col]) * 1.25)
for ax, ref in [(axes[0], df.fire.mean() * 100), (axes[1], (df.area > 10).mean() * 100)]:
    ax.set_xlim(-0.5, 3.0)
    ax.text(2.35, ref, f" overall\n {ref:.0f}%", fontsize=9, color=INK2, va="center")
fig.suptitle("Risk tiers: the High tier burns more often than the Low tier (60% vs 42%) — but the model cannot single out large fires",
             x=0.01, ha="left", fontsize=13, fontweight="bold", y=1.04)
save(fig, "fig12_risk_tiers.png")
```

```
Out-of-fold ROC-AUC: 0.592
             n  fire_rate  large_rate  burned_ha
risk_tier
Low        171       42.1        18.1     1850.9
Moderate   171       55.6        18.7     3387.8
High       171       59.6        18.1     1374.7
```

Figure 20: Notebook: cross-validated ROC-AUC of the four models and the out-of-fold risk-tier table

5. Conclusion

5.1 Summary of findings

Visual storytelling turned 513 rows of weather records into a clear picture: Montesinho's fire problem is **seasonal** (August–September hold 81% of burned hectares), **concentrated** (the 10 largest fires cause 51% of the damage and two grid cells 40%) and **driven by accumulated dryness and heat**. The Plotly Dash dashboard lets officers explore these patterns interactively, and the Random Forest risk tiers (cross-validated ROC-AUC 0.60) give a modest but real way to rank days by fire likelihood. Together they support a shift from reactive fire-fighting to planned, data-driven prevention.

5.2 Limitations

- Small dataset (513 observations) from one park and four years; 3 months have fewer than 5 observations.
- Weather explains **when** danger is high but not **which exact plot** will burn — ignition sources (people, lightning) are not in the data. This matches the original study (Cortez & Morais, 2007).
- Burned area is extremely skewed, so averages are pulled up by a few huge fires; medians and log scales are used where appropriate.

5.3 Future scope

- Add ignition-related data (distance to roads and villages, tourist counts, lightning strikes) to improve the model.
- Connect the dashboard to a live weather feed so the risk check runs automatically every morning.
- Extend the analysis to more parks and years to test whether the hotspots and thresholds generalise.

6. References

1. Cortez, P. & Morais, A. (2007). A Data Mining Approach to Predict Forest Fires using Meteorological Data. Proc. 13th EPIA – Portuguese Conference on Artificial Intelligence, pp. 512–523.
2. UCI Machine Learning Repository — Forest Fires dataset: https://archive.ics.uci.edu/dataset/162/forest+fires
3. Anasse Bari, Mohamed Chaouchi, Tommy Jung — Predictive Analytics For Dummies, 2nd Ed., Wiley, 2016.
4. Alexandru Telea — Data Visualization: Principles and Practice, 2nd Ed., CRC Press, 2014.
5. Brett Lantz — Machine Learning with R: Expert techniques for predictive modeling, 3rd Ed., 2019.
6. Chun-houh Chen, Wolfgang Härdle, Antony Unwin — Handbook of Data Visualization, Springer, 2008.
7. Plotly Dash documentation: https://dash.plotly.com
8. scikit-learn documentation: https://scikit-learn.org

Appendix — Submission files

File / folder	Contents
Activity2_Data_Storytelling.ipynb	Full analysis: cleaning, 10 visualizations, model, insights (executed, with outputs)
dashboard/app.py, figures.py, assets/	Interactive Plotly Dash dashboard
dashboard/Wildfire_Dashboard.html	Offline static copy of the dashboard
data/	Raw dataset, attribute description and cleaned CSV
figures/	All charts and screenshots as PNG
Activity2_Report.docx / .pdf	This report
Activity2_Presentation.pptx / .pdf	Presentation slides